



CineMax - Manuale Tecnico e Specifiche Architetture Avanzate

Università degli Studi dell'Insubria

Dipartimento di Scienze Teoriche e Applicate

Laboratorio Interdisciplinare A (Anno Accademico 2025/2026)

Docente Referente: Prof. Giovanni Meroni

Gruppo di Sviluppo:

- Rossetti Andrea (765635 - VA)
- Segato Alessandro (766931 - VA)
- Bergamo Emma (766925 - VA)
- Piccolo Matteo (765213 - VA)

Data di Emissione: Settembre 2026

Indice Generale dei Contenuti Tecnici

- [1. Introduzione e Specifiche di Sistema](#)
- [2. Architettura del Software e Scelte Sistemistiche](#)
- [3. Analisi Dettagliata dei Package e delle Classi](#)
- [4. Strutture Dati e Gestione delle Collezioni in Memoria](#)
- [5. Specifiche Algoritmiche e Regole di Business](#)
- [6. Formato dei File, Path di Persistenza e Gestione I/O](#)
- [7. Pattern di Progettazione e Snippet di Codice](#)
- [8. Standard Javadoc e Documentazione del Codice](#)

1. Introduzione e Specifiche di Sistema

Il presente Manuale Tecnico fornisce una trattazione approfondita, rigorosa e dettagliata della struttura implementativa, delle scelte architetture, dei pattern di progettazione e delle specifiche algoritmiche adottate nello sviluppo del sistema software CineMax. Il progetto è stato realizzato nell'ambito del corso di Laboratorio Interdisciplinare A presso l'Università degli Studi dell'Insubria (Dipartimento di Scienze Teoriche e Applicate).

L'applicazione nasce per digitalizzare e gestire in modo centralizzato le attività operative di un cinema monosala con una capienza fissa di 200 posti. Il sistema supporta molteplici ruoli utente (Guest, Cliente, Bigliettaio, Proiezionista), garantendo un controllo rigoroso degli accessi, la persistenza sicura delle informazioni e un'interfaccia interattiva testuale (TUI) altamente ottimizzata.

Comando di Esecuzione Ufficiale: L'applicazione è distribuita sotto forma di archivio eseguibile ed è testata per l'esecuzione tramite Java Development Kit / Runtime Environment (JDK/JRE 17 o superiore). Il comando da terminale per l'avvio è:

```
java -jar bin/CineMax.jar
```

I requisiti minimi di sistema sono estremamente contenuti in termini di memoria RAM e spazio su disco, rendendo CineMax una soluzione leggera, portatile e indipendente da DBMS relazionali complessi a server remoto.

2. Architettura del Software e Scelte Sistemistiche

L'architettura software di CineMax è stata concepita applicando i principi fondamentali dell'ingegneria del software orientata agli oggetti. In particolare, è stata perseguita una netta separazione delle responsabilità (Separation of Concerns), articolando l'applicazione in differenti layer funzionali:

2.1 Stratificazione dei Layer

- Layer di Presentazione (TUI):** Localizzato nella classe principale `CineMax`, gestisce interamente l'interazione testuale con l'utente tramite console. Si occupa di stampare i menu formattati, acquisire gli input da tastiera tramite Scanner e validare preliminarmente i dati prima di invocarne la logica sottostante.
- Layer di Logica Applicativa (Facade / Controller):** Incapsulato nella classe `Sistema`, rappresenta il punto di smistamento e coordinamento centrale. Gestisce i flussi operativi per ciascun ruolo, l'autenticazione, la validazione delle regole di business e la transazionalità delle operazioni in memoria.
- Layer di Persistenza (Data Access):** Implementato nella classe di utilità `GestoreDati`, centralizza tutte le operazioni di lettura e scrittura su file system, gestendo l'inizializzazione della cartella locale `data/` e il parsing robusto dei file TXT e CSV.
- Layer di Dominio (Model):** Raggruppa le classi che definiscono le entità fondamentali del dominio cinematografico e anagrafico (`Utente`, `Cliente`, `Bigliettaio`, `Proiezionista`, `Film`, `Proiezione`, `Prenotazione`).

2.2 Flusso dei Dati tra i Componenti

All'avvio dell'applicazione, il layer di persistenza legge i file memorizzati sul disco e popola le collezioni in memoria all'interno del layer di business. L'utente interagisce unicamente con la TUI, la quale inoltra le richieste alla Facade (`Sistema`). Quest'ultima elabora la logica di business e, se necessario, ordina al layer di persistenza di aggiornare i file su disco. Tale approccio garantisce che la TUI sia completamente disaccoppiata dai meccanismi di memorizzazione.

3. Analisi Dettagliata dei Package e delle Classi

Il codice sorgente dell'applicazione è organizzato all'interno del package radice `cinemax`. Di seguito viene presentata un'analisi tecnica dettagliata di ciascuna classe che compone il sistema:

Classe Java	Tipologia / Visibilità	Responsabilità e Ruolo Architetturale
<code>CineMax.java</code>	Public Class (Driver)	Contiene il metodo <code>main</code> e gestisce il loop applicativo principale (<code>avvia()</code>), la visualizzazione dei loghi ASCII e lo smistamento ai menu specifici per ruolo.
<code>Sistema.java</code>	Public Class (Facade)	Gestisce lo stato corrente dell'utente loggato, coordina le liste di proiezioni, utenti e prenotazioni, implementando i metodi operativi richiesti dai menu.
<code>GestoreDati.java</code>	Public Utility Class	Responsabile dell'I/O su file. Effettua la deserializzazione dei file TXT e il parsing avanzato dei file CSV (gestendo virgolette e delimitatori complessi).
<code>Utente.java</code>	Public Abstract Class	Modella l'utente generico del sistema. Implementa la cifratura delle password tramite algoritmo SHA-256 (MessageDigest) e

Classe Java	Tipologia / Visibilità	Responsabilità e Ruolo Architettuale
		gestisce l'anagrafica comune.
Cliente.java	Public Subclass	Estende Utente; rappresenta il cliente registrato abilitato all'effettuazione e gestione autonoma delle prenotazioni in sala.
Bigliettaio.java	Public Subclass	Estende Utente; abilita il personale di cassa alla ricerca avanzata delle prenotazioni e alla visualizzazione degli ingressi odierni.
Proiezionista.java	Public Subclass	Estende Utente; consente la gestione del catalogo film, l'inserimento di nuove proiezioni e la pianificazione del palinsesto.
Film.java	Public Domain Class	Modella l'opera cinematografica memorizzando titolo, genere, regista, anno di uscita, durata in minuti e età minima consigliata per la visione.
Proiezione.java	Public Domain Class	Associa un oggetto Film a una data/ora specifica (LocalDateTime), a un costo del biglietto e alla capienza della sala (fissata a 200 posti).
Prenotazione.java	Public Domain Class	Associa un Cliente a una Proiezione, memorizzando il numero di biglietti, il costo totale e generando un codice UUID univoco a 8 caratteri.

4. Strutture Dati e Gestione delle Collezioni in Memoria

Per garantire un accesso rapido ed efficiente alle informazioni durante l'esecuzione, il sistema fa ricorso alle collezioni standard del Java Collections Framework, evitando l'overhead di connessioni di rete o query SQL complesse:

4.1 Utilizzo di ArrayList e incapsulamento

Tutte le entità persistenti caricate da file vengono mantenute in memoria all'interno di istanze di `java.util.ArrayList`:

- `List<Proiezione> proiezioni`: Conserva l'intero palinsesto cinematografico. L'accesso sequenziale consente di eseguire ricerche filtrate per intervalli di date o genere con complessità temporale lineare $O(n)$.
- `List<Utente> utenti`: Mantiene in memoria l'anagrafica di tutti gli utenti autorizzati ad accedere al sistema, permettendo la validazione istantanea delle credenziali in fase di login.
- `List<Prenotazione> prenotazioni`: Raccoglie tutte le prenotazioni effettuate dai clienti, fungendo da tabella di associazione relazionale in-memory tra utenti e proiezioni.

4.2 Gestione Temporale e Identificativi Univoci

Il sistema sfrutta le API moderne di Java 8+ per la gestione del tempo e degli ID:

- `java.time.LocalDate` e `java.time.LocalDateTime`: Utilizzate per gestire con precisione millimetrica la pianificazione delle proiezioni, evitando ambiguità nei fusi orari e semplificando i confronti temporali (es. verifica proiezioni future o odierne).
- `java.util.UUID`: Utilizzato nella classe Prenotazione per generare codici alfanumerici univoci a 8 caratteri (es. 4A8F9B12), garantendo l'assenza di collisioni tra i biglietti emessi.

5. Specifiche Algoritmiche e Regole di Business

La logica algoritmica implementata all'interno del layer di business risponde a rigidi vincoli di coerenza e integrità funzionale:

5.1 Algoritmo di Ricerca e Filtraggio Multi-parametro

Le funzioni di ricerca avanzata (come `cercaProiezioni` e `cercaPrenotazioni`) implementano un pattern di filtraggio condizionale sequenziale. Quando l'utente inserisce parametri opzionali (titolo, genere, date di inizio e fine, range di prezzo), il sistema scorre la lista in memoria applicando predicati booleani cumulativi. I campi lasciati vuoti vengono automaticamente ignorati, garantendo flessibilità e robustezza contro input parziali.

5.2 Algoritmo di Controllo della Capienza e Posti Liberi

Il cinema gestito da CineMax ha una capienza fissa di 200 posti per sala. Il controllo della disponibilità e il calcolo dei posti liberi per una determinata proiezione vengono eseguiti dinamicamente tramite un algoritmo di aggregazione:

```
public int calcolaPostioccupati (int idProiezione) {
    int occupati = 0;
    for (Prenotazione p : prenotazioni) {
        if (p.getProiezione().getId() == idProiezione) {
            occupati += p.getNumeroBiglietti();
        }
    }
    return occupati;
    // Posti Liberi = Capacità Sala (200) - postiOccupati
}
```

Prima di confermare qualsiasi nuova prenotazione o modifica, il sistema verifica che `postiLiberi >= numeroBigliettiRichiesti`, rifiutando l'operazione in caso contrario.

5.3 Algoritmo di Hashing Crittografico (SHA-256)

Per garantire la sicurezza delle credenziali, le password non vengono mai salvate in chiaro. La classe `Utente` implementa un algoritmo basato sulla classe `java.security.MessageDigest`:

- Viene istanziato l'algoritmo SHA-256.
- La password in chiaro viene convertita in array di byte e sottoposta a digest crittografico.
- L'array risultante viene formattato in una stringa esadecimale a 64 caratteri, confrontata direttamente con l'hash memorizzato in `utenti.txt`.

6. Formato dei File, Path di Persistenza e Gestione I/O

Il sistema garantisce la persistenza delle informazioni su file system locale all'interno della directory di lavoro. All'avvio dell'applicazione, un blocco statico di inizializzazione all'interno della classe `GestoreDati` verifica l'esistenza della cartella di servizio e la crea se assente:

```
private static final String DIR_DATA = "data";
private static final String FILE_PROIEZIONI = "data/proiezioni.txt";
private static final String FILE_UTENTI = "data/utenti.txt";
private static final String FILE_PRENOTAZIONI = "data/prenotazioni.csv";

static {
```

```

File dir = new File(DIR_DATA);
if (!dir.exists()) {
    dir.mkdirs();
}
}

```

6.1 Specifiche dei Tracciati di Record

- `utenti.txt` : File di testo strutturato con record delimitati da punto e virgola (;). Ciascuna riga memorizza: nome; cognome; username; passwordHash; dataNascita; luogoDomicilio; ruolo . I ruoli ammessi sono cliente, bigliettaio e proiezionista.
- `proiezioni.txt` : File di testo formattato con separatore che memorizza l'ID numerico della proiezione, i dati anagrafici del film (titolo, genere, regista, anno, durata, età minima), la data e ora nel formato standard e il costo del biglietto.
- `prenotazioni.csv` : File in formato tabulare delimitato da virgole utilizzato per memorizzare le relazioni di acquisto: usernameCliente, idProiezione, numeroBiglietti .

6.2 Gestione dei File CSV di Fallback

In assenza di un file di proiezioni locale, `GestoreDati` implementa un meccanismo di ricerca automatica nei percorsi di fallback (inclusi file CSV di test forniti con il progetto), eseguendo il parsing riga per riga e convertendo i dati nel formato nativo del sistema.

7. Pattern di Progettazione e Snippet di Codice

Lo sviluppo del codice sorgente ha fatto ricorso a pattern di progettazione software riconosciuti:

- **Facade Pattern:** La classe `Sistema` unifica e nasconde la complessità delle collezioni sottostanti e dei moduli di I/O, esponendo un'interfaccia pulita e coesa utilizzata dal layer di presentazione.
- **Inheritance & Polymorphism Pattern:** La classe astratta `Utente` definisce gli attributi comuni e il contratto di cifratura, consentendo alle sottoclassi di ereditare i comportamenti e specializzare le interazioni.

Snippet Significativo 1: Logica di Hashing SHA-256 (Utente.java)

```

private String hashPassword (String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hashedBytes = md.digest(password.getBytes());
        StringBuilder sb = new StringBuilder();
        for (byte b : hashedBytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("Errore critico nel calcolo dell'hash della password", e);
    }
}

```

Snippet Significativo 2: Parsing di Linea CSV con gestione virgolette (GestoreDati.java)

```

private static String[] parseCSVLine (String line) {
    List<String> risultato = new ArrayList<>();
    StringBuilder campoCorrente = new StringBuilder();
    boolean dentroVirgolette = false;

    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (c == '"') { dentroVirgolette = !dentroVirgolette; }
        else if (c == ',' && !dentroVirgolette) {
            risultato.add(campoCorrente.toString());
            campoCorrente = new StringBuilder();
        } else { campoCorrente.append(c); }
    }
    risultato.add(campoCorrente.toString());
    return risultato.toArray(new String[0]);
}

```

8. Standard Javadoc e Documentazione del Codice

L'intero codice sorgente del progetto CineMax è rigorosamente commentato in conformità con lo standard Javadoc stabilito da Oracle. Ogni pacchetto, classe, costruttore, attributo e metodo pubblico espone intestazioni descrittive che includono:

- Una descrizione sintetica e approfondita dello scopo del componente.
- Il tag `@author` per l'attribuzione del codice agli sviluppatori del gruppo (Rossetti, Segato, Bergamo, Piccolo).
- Il tag `@param` per la descrizione dettagliata di ciascun parametro in ingresso.
- Il tag `@return` per la specifica del valore restituito dai metodi non void.